

第二阶段个人工作小结

1、本阶段工作内容

这几周项目进入了核心代码开发部分，我的工作重心也从前期的搭环境变成了死磕具体的算法与业务逻辑。这段时间我主要把基于大模型的 RAG（检索增强生成）核心主流程给写完了。结合上阶段预留的数据库底层接口，我实现了对教师上传教案的自动文本切片，并成功调用了向量模型将文本转化为高维向量，持久化到了 PostgreSQL（pgvector）中。在接口层面，我把 chat_routes.py 里的对话路由全部补齐，并且深度优化了 deepseek_client.py 里面的请求逻辑，接入了流式输出（Streaming），实现了类似真实 ChatGPT 的打字机回复效果。此外，这几周我还一直在和谢巧进行后端双服务的内部联调，确保他写的 Java 业务网关能准确地把前端流量和参数转发到我这个 Python 算法容器里，终于从请求接入到向量检索，再到组装 Prompt 和调用大模型，到最后结果流式返回的这条最核心的数据链路给彻底跑通了。

2、遇到的问题以及解决思路

在这阶段堆代码的过程中踩了不少坑，排查最久、印象最深的有两个问题。

一个是 大模型上下文超限与 RAG 检索的幻觉干扰。刚开始测试知识库问答的时候，图省事，我直接把用户提问和数据库里查出来的相关教案段落一股脑全塞进 Prompt 里丢给 DeepSeek。结果发现在提问稍微复杂一点时，要么 API 直接报 Token 超限的错误截断了连接，要么就是查出来的段落太多太杂，大模型被无关信息干扰，开始脱离教案胡编乱造。

为了解决这个问题，我彻底重构了 Python 端的切片与检索逻辑。首先，引入了基于“滑动窗口”的 Chunking（切片）算法，保证切出来的每段文字既不太长也不会丢失上下文连贯性。其次，在 db.py 里面优化了 SQL 的检索策略，加上了余弦相似度的严格阈值过滤，并强行限制只取最相关的三段。最后，我还调优了发给大模型的 System Prompt，用强指令约束大模型只能基于给定的参考内容进行回答。最终这一整套操作下来，准确率和接口响应速度都稳住了。

第二个印象比较深的问题是 Java 与 Python 容器联调时的 Docker 内部网络故

障。

在我和谢巧第一次把各自的服务打好镜像扔进 docker-compose 准备联调时，他的 Java 服务只要一调我的算法接口，就疯狂报 Connection Refused。但非常诡异的是，我在宿主机用 Postman 去请求我暴露出来的端口，一切返回都是正常的。我们查了一下午的防火墙和接口路径，都没找到原因。

后来仔细看网络文档才反应过来是容器隔离的坑。我们在本地写代码时，习惯性地吧请求地址写死成 localhost:127.0.0.1。但在 Docker 环境下，Java 容器的 localhost 和 Python 容器的 localhost 不是一回事。找到原因后，我马上把 Python 启动脚本里绑定的 IP 从 127.0.0.1 改成了 0.0.0.0 允许外部访问，然后让谢巧把 Java 代码里的 HTTP 请求域名替换成了我们在 docker-compose.yml 里定义的 Python 服务名（利用 Docker 内部的 DNS 解析）。改完重启容器，两边的数据总算能顺畅互通了。

3、下一阶段计划

目前底层的对话逻辑已经比较完善了，下个阶段的主要任务是要结合我们系统的主打功能动态可视化渲染，去做深层的结构化数据清洗。我计划重点打磨大模型的输出格式，编写正则表达式和 JSON 校验拦截器，确保大模型在输出讲解文字的同时，能够稳定地吐出符合前后端约定好的 DSL 动画指令协议。同时，要开始密集地和前端对接，调优动态渲染图谱的显示效果。如果时间充裕，还会着手写一些自动化压测脚本，测一测并发情况下的表现。